

# Respostas Conceituais

Lista Git, GitHub e GitFlow

|             |                                                                                                       |
|-------------|-------------------------------------------------------------------------------------------------------|
| Disciplina  | Engenharia da Qualidade e Confiabilidade                                                              |
| Aluno       | Sam280903                                                                                             |
| Data        | 11/06/2026                                                                                            |
| Repositório | <a href="https://github.com/Sam280903/lanchonete-web">https://github.com/Sam280903/lanchonete-web</a> |

## PARTE 1 — Fundamentos do Git

### Questao 1c — O que significa "Untracked files"?

"Untracked files" indica que o Git detectou arquivos na pasta do projeto que **ainda nao estao sendo monitorados** por ele. Esses arquivos existem no diretorio de trabalho, mas nenhum *git add* foi realizado sobre eles. O Git os exibe como aviso para que o desenvolvedor decida se deseja inclui-los no controle de versao.

### Questao 2a — O que mudou apos *git add README.md*?

Apos o *git add README.md*, o arquivo saiu da secao "**Untracked files**" e passou para a secao "**Changes to be committed**" (Staging Area). Isso significa que o README.md esta preparado (staged) para ser incluido no proximo commit. Os demais arquivos continuam como "Untracked".

### Questao 2d — O que *git log --oneline* exibe?

Exibe o historico de commits de forma resumida, com **uma linha por commit**, mostrando o **hash curto** (7 caracteres) e a **mensagem do commit**. E ideal para ter uma visao rapida da linha do tempo do projeto.

### Questao 3a — Diferenca entre Untracked e Modified

| Status    | Significado                                                                                       |
|-----------|---------------------------------------------------------------------------------------------------|
| Untracked | Arquivo novo que o Git nunca rastreou. Nao existe em nenhum commit anterior.                      |
| Modified  | Arquivo que o Git ja conhece (ja foi commitado), mas que sofreu alteracoes desde o ultimo commit. |

### Questao 3b — O que *git diff* mostra?

O *git diff* mostra as **diferencas linha a linha** entre o arquivo no diretorio de trabalho e a versao na Staging Area (ou no ultimo commit). Linhas com **+** sao adicoes; com **-** sao remocoes. Permite revisar exatamente o que sera incluido antes do *git add*.

### Questao 4a — Informacoes do git log (versao completa)

O *git log* exibe para cada commit:

- **Hash completo** — identificador SHA-1 unico do commit (40 caracteres)
- **Author** — nome e e-mail de quem realizou o commit
- **Date** — data e hora do commit
- **Mensagem** — descricao do que foi alterado

### Questao 4b — O simbolo \* no git log --oneline --graph

O simbolo \* representa **um commit na linha do tempo**. Cada asterisco e um ponto no grafo do historico. Quando ha branches, o grafo exibe ramificacoes (|\\, |/) para mostrar onde os caminhos divergiram e convergiram.

### Questao 4c — Importancia de mensagens de commit claras

- **Facilitam a revisao de codigo** — qualquer desenvolvedor entende o que foi feito sem analisar o diff.
- **Agilizam a depuracao** — e possivel identificar rapidamente qual commit introduziu um bug.
- **Documentam o historico** — o git log vira um diario do projeto com decisoes tecnicas rastreadas.
- **Melhoram a colaboracao** — em equipes, commits bem descritos evitam retrabalho e comunicacao redundante.

### Questao 5c — O que o asterisco (\*) em git branch indica?

O asterisco indica a **branch atual** (na qual o HEAD esta apontando). E a branch que recebera os proximos commits.

### Questao 5d — Por que promocoos.html desapareceu ao voltar para main?

Porque o arquivo foi commitado **apenas na branch feature/pagina-promocoos**, nao na main. O Git gerencia o estado de cada branch de forma independente — ao trocar de branch, o diretorio de trabalho reflete exatamente os arquivos daquela branch. Como o promocoos.html nao existia na main, ele simplesmente nao aparece.

### Questao 6b — Estrategia de merge utilizada

O Git utilizou a estrategia **Fast-forward**. Ela ocorre quando a branch de destino (main) nao teve nenhum commit novo desde a criacao da branch de feature. O Git simplesmente avanca o ponteiro da main para o commit mais recente da feature, sem criar um commit de merge explicito.

### Questao 6c — O que mudou no historico apos o merge?

No *git log --oneline --graph*, o historico permaneceu linear (sem bifurcacao visivel) por causa do Fast-forward. O ponteiro main foi simplesmente avancado para incluir o commit da feature. Nao ha commit de merge explicito.

### Questao 6d — Por que deletar branches concluidas?

- **Limpeza do repositorio** — evita acumulo de branches obsoletas.
- **Clareza de proposito** — branches ativas devem representar trabalho em andamento.
- **Reducao de conflitos futuros** — branches antigas acumulam divergencias desnecessarias.

### Questao 7c — Diferenca entre git revert e git reset

| Comando    | Comportamento                                                                            |
|------------|------------------------------------------------------------------------------------------|
| git revert | Cria um NOVO commit que desfaz as mudancas do commit indicado. O historico e preservado. |
| git reset  | Move o ponteiro do HEAD para um commit anterior, podendo apagar commits do historico.    |

**Por que revert e mais seguro em projetos colaborativos?** Porque ele **nao reescreve o historico**. Em projetos com varios colaboradores, usar *git reset* em commits ja enviados ao remoto causaria divergencias graves no historico de outros desenvolvedores. O revert adiciona transparencia: fica registrado que uma mudanca foi desfeita e por que.

### Questao 8c — O que significa a flag -u no git push?

A flag **-u** (`--set-upstream`) configura a branch local para **rastrear a branch remota** correspondente. Apos usar *git push -u origin main*, nos proximos pushes basta digitar *git push*, pois o Git ja sabe para onde enviar.

### Questao 8d — O que o git pull fez?

O *git pull* executou dois comandos em sequencia:

- **git fetch** — baixou as alteracoes do repositorio remoto.
- **git merge** — integrou essas alteracoes na branch local atual.

A alteracao feita diretamente no GitHub (no README.md) passou a existir tambem no repositorio local.

---

## PARTE 2 — GitFlow

---

### Questao 9c — Qual branch foi criada alem da main?

A branch **develop** foi criada. Ela serve como a **linha de integracao continua** do projeto — e onde todas as features sao incorporadas antes de irem para producao. A main representa apenas versoes estaveis e liberadas; a develop e o ramo de desenvolvimento ativo.

### Questao 10a — Em qual branch git flow feature start coloca?

Na branch **feature/cardapio-interativo**, criada automaticamente a partir da **develop**.

### Questao 10c — O que git flow feature finish fez automaticamente?

- Fez o **merge** da branch feature/cardapio-interativo na develop.
- **Deletou** a branch de feature localmente.
- Voltou para a branch **develop**.

O código foi enviado para a branch **develop** (não para a main).

### Questão 11a — Em qual branch o release é criado e a partir de qual?

A branch de release é criada como **release/1.0.0**, nascendo a partir da branch **develop**. Ela captura o estado atual do desenvolvimento e isola os ajustes finais de preparação para o lançamento.

### Questão 11c — O que aconteceu com main e develop após release finish?

- Fez **merge da release na main** — a versão estável foi lançada.
- Criou a **tag 1.0.0** na main.
- Fez **merge da release de volta na develop** — para que os ajustes finais também existam no desenvolvimento.
- Deletou a branch release/1.0.0.

### Questão 11d — Função de uma tag no GitFlow

Tags marcam **pontos imutáveis no histórico**, representando versões liberadas para produção. No GitFlow, cada release finish cria uma tag (ex: 1.0.0) para que qualquer versão passada possa ser recuperada exatamente como foi entregue. São análogas a "fotografias" do estado do software.

### Questão 12a — Por que o hotfix nasce da main e não da develop?

Porque o hotfix precisa corrigir um problema que está **em produção agora**, e a main representa exatamente o estado em produção. Se nascesse da develop, o hotfix incluiria features não liberadas, contaminando a correção urgente com código não validado.

### Questão 12c — Para quais branches o hotfix foi mesclado?

- **main** — para corrigir o problema em produção imediatamente.
- **develop** — para que a correção também esteja presente no desenvolvimento futuro.

### Questão 12d — Número de versão após o hotfix (SemVer)

Seguindo o **Versionamento Semântico (SemVer)** no formato MAJOR.MINOR.PATCH:

| Campo        | Quando incrementa                             |
|--------------|-----------------------------------------------|
| <b>MAJOR</b> | Mudanças incompatíveis com versões anteriores |
| <b>MINOR</b> | Novas funcionalidades retrocompatíveis        |
| <b>PATCH</b> | Correções de bugs retrocompatíveis            |

Um hotfix é uma correção de bug, portanto incrementa o **PATCH**: **1.0.0** → **1.0.1**

### Questao 13b — Quando usar GitFlow (e quando nao usar)

**GitFlow e mais indicado em:**

- Projetos com ciclos de release planejados (ex: versoes mensais).
- Equipes maiores com multiplos desenvolvedores em features paralelas.
- Softwares com multiplas versoes em producao simultaneamente.

**GitFlow seria desnecessariamente complexo em:**

- Projetos pequenos ou solo, como aplicacoes pessoais ou exercicios academicos.
- Times com entrega continua (CI/CD), onde codigo vai a producao varias vezes ao dia.
- Prototipos e MVPs onde a velocidade e mais importante que a formalidade.

### Questao 13c — Comparacao: sem GitFlow vs. com GitFlow

| Aspecto     | Sem GitFlow (Q1-8) / Com GitFlow (Q9-13)                                                                    |
|-------------|-------------------------------------------------------------------------------------------------------------|
| Vantagem    | Com GitFlow: historico organizado e semantico. Fica claro o que e feature, release ou hotfix. A separacao o |
| Desvantagem | Com GitFlow: adiciona complexidade desnecessaria para projetos simples. Muitas branches, merges e tag       |